



Universidad
Andrés Bello®

Modelado, Implementación y Análisis de Clique Cover

Análisis de Problemas NP-Completo | Facultad de
Ingeniería

Matias Santos • Jose Romero • David Cortez

Profesor: Luis Fuentes Larenas

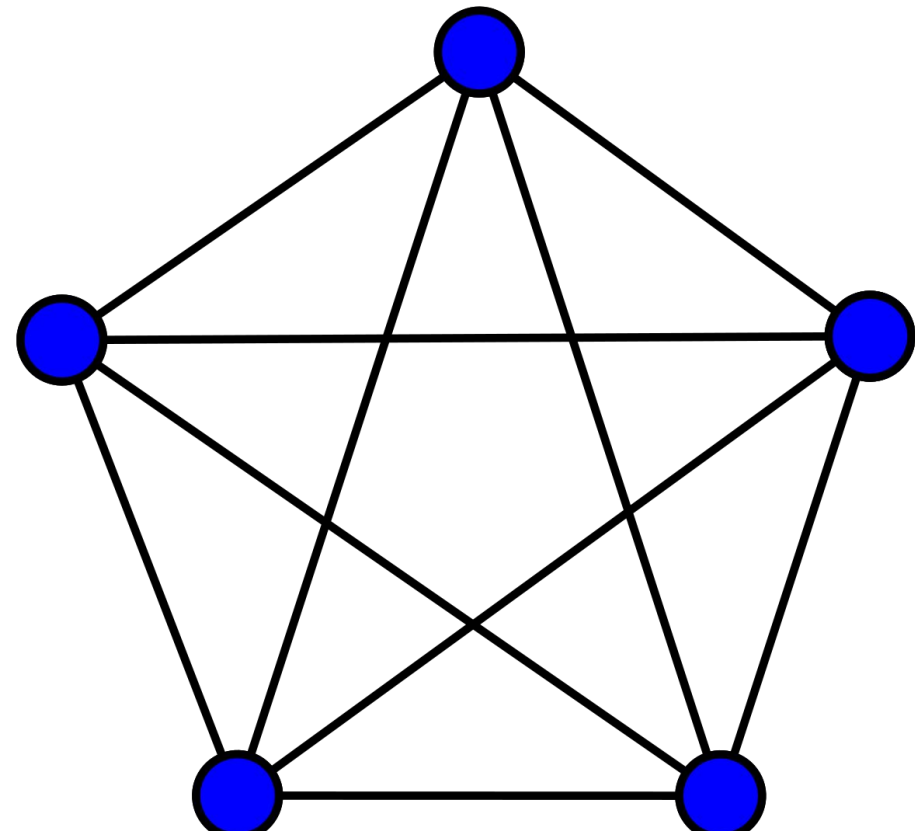
Santiago, Chile | 2026

1. EL PROBLEMA: CLIQUE COVER

🔍 **Definición:** Particionar los vértices de un grafo $G=(V,E)$ en el mínimo número k de subconjuntos.

🔗 **Condición:** Cada subconjunto debe ser un *clique* (todos sus vértices están conectados entre sí).

🧩 **Ejemplo:** En un grafo de 4 nodos donde $\{1,2,3\}$ están conectados, pero el 4 solo con el 3, el resultado es $k=2$.



2. FORMULACIÓN MATEMÁTICA

Variables de Decisión

✓ x_{ik} : 1 si el vértice i está en el clique k y 0 en el caso contrario

≡ y_k : 1 si el clique k es utilizado y 0 en el caso contrario.

Restricciones y Objetivo

Minimizar:

$$\min \sum_{k \in K} y_k$$

Sujeto a: Cada vértice en 1 clique y nodos no adyacentes no comparten clique.

3. COMPLEJIDAD COMPUTACIONAL



Pertenencia a NP

Dada una partición de vértices, verificar si cada subconjunto es un clique se realiza en tiempo polinomial $O(V^2)$.



NP-Duro

Se demuestra mediante reducción desde **Graph Coloring**. Cubrir G con k cliques es equivalente a colorear el grafo complemento $\bar{G}$.



Veredicto

Al ser NP y NP-Duro, Clique Cover es **NP-Completo**. No existe algoritmo polinomial conocido para la solución óptima general.

4. ESTRATEGIAS DE IMPLEMENTACIÓN

Base: Backtracking

Exploración sistemática del espacio de soluciones. Garantiza la **optimalidad** pero con costo exponencial.

Uso de Branch & Bound en C++.

Mejora: Greedy

Estrategia ávida. Asigna vértices a cliques existentes minimizando nuevas creaciones. Extremadamente **rápido**.

Ahorro de tiempo masivo.

Decisiones Clave

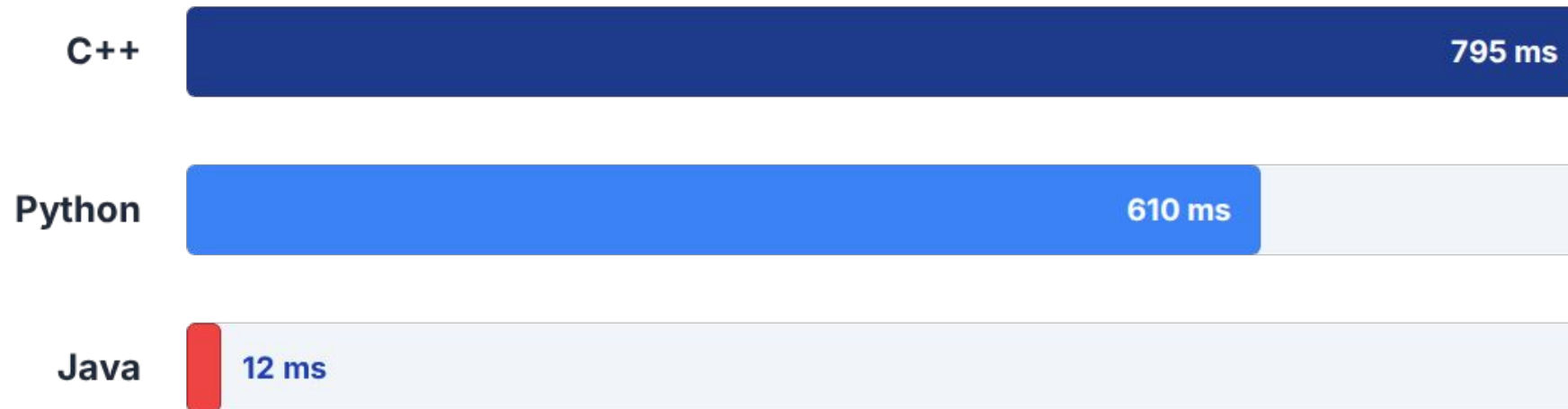
C++: Optimización a bajo nivel (Bitsets).

Java: Terminación temprana y JIT.

Python: Parada temprana pero mayor overhead.

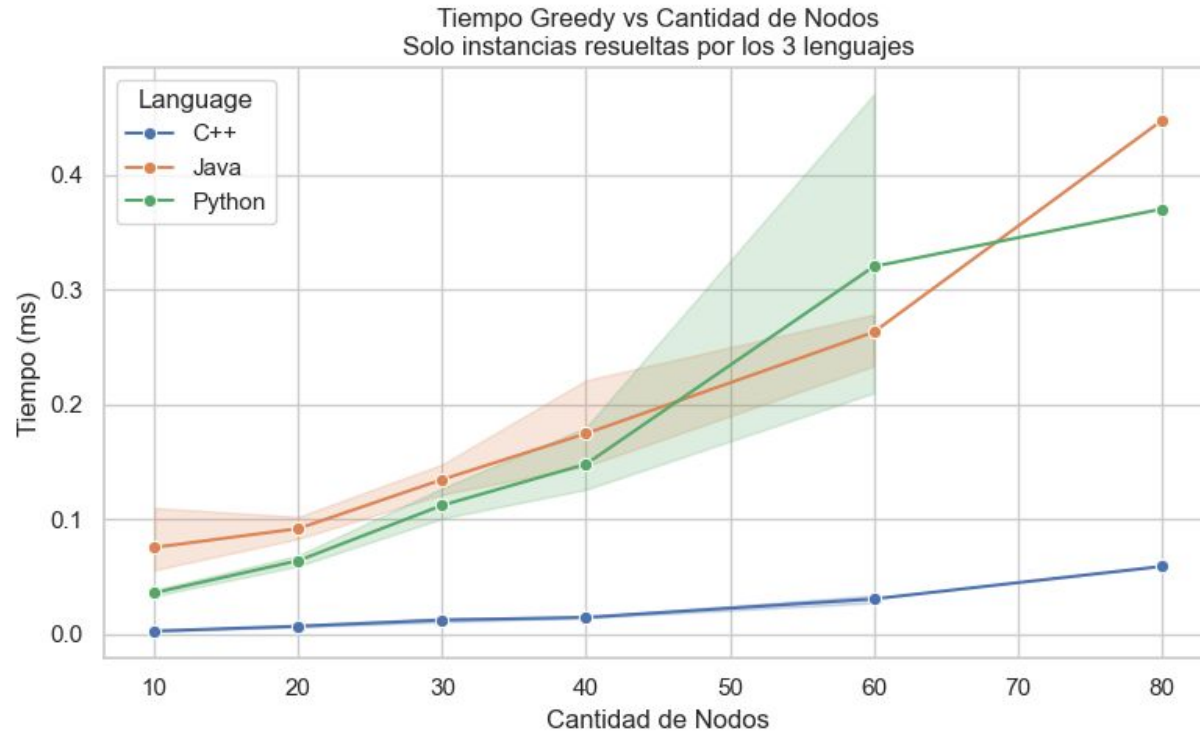
5. ANÁLISIS: SOLUCIÓN BACKTRACKING

Comparación de tiempos de ejecución promedio (53 instancias) para obtener la solución **óptima**.



Nota: El enfoque de terminación temprana en Java/Python superó a la exploración exhaustiva de C++ en tiempo real.

ANÁLISIS: ESCALABILIDAD GREEDY



Crecimiento lineal de la solución Greedy.

Trade-off: Calidad vs Tiempo

Greedy reduce el tiempo a casi 0ms (C++: 0.01ms), pero introduce una penalización en la calidad:

- **C++ / Java:** +1.1 cliques extras.
- **Python:** +1.8 cliques extras (fallo de heurística de grado).

6. CONCLUSIONES Y APRENDIZAJES

-  **Complejidad Real:** Los problemas NP-completos exigen un compromiso entre optimalidad y tiempo de respuesta.
-  **Lenguajes:** C++ es superior para heurísticas de alto rendimiento; Java destaca en Backtracking con JIT.
-  **Mejora:** La terminación temprana es vital en Backtracking, mientras que el ordenamiento en Greedy debe ser cuidadosamente validado.

Muchas gracias por su atención.